

Computer Graphics Final Project 2026

Report

111590453 資工四 張竣崴

Theme: AI Future World – Palmanova Star City

i. Project Setup

This section explains how to compile and run the project.

- **Prerequisites:** You must have a C++ Compiler (g++), GNU Make, and FreeGLUT installed on your system.
- **Configuration:** The project expects FreeGLUT to be located at `C:\freeglut` by default. If your installation is elsewhere, you must edit lines 3–4 of the Makefile to update the include and library paths.
- **Compilation:** Open your terminal and run `make` to compile the project. By default, this compiles the project without Xbox controller support (`ENABLE_CONTROLLER=0`) to eliminate unnecessary headers and library dependencies.
- **Execution:** Run `make run` to launch the application.
- **Xbox Controller Support:**
 - To compile and run with Xbox controller support enabled: run `make control` (which cleans, recompiles with the `ENABLE_CONTROLLER=1` macro, and launches the app).
 - Alternatively, to just build with controller support enabled, run `make ENABLE_CONTROLLER=1`.
- **Cleanup:** Run `make clean` to clear build files.

ii. User Guide

The application supports both Keyboard controls and Windows XInput Gamepad controls for navigating the 3D city.

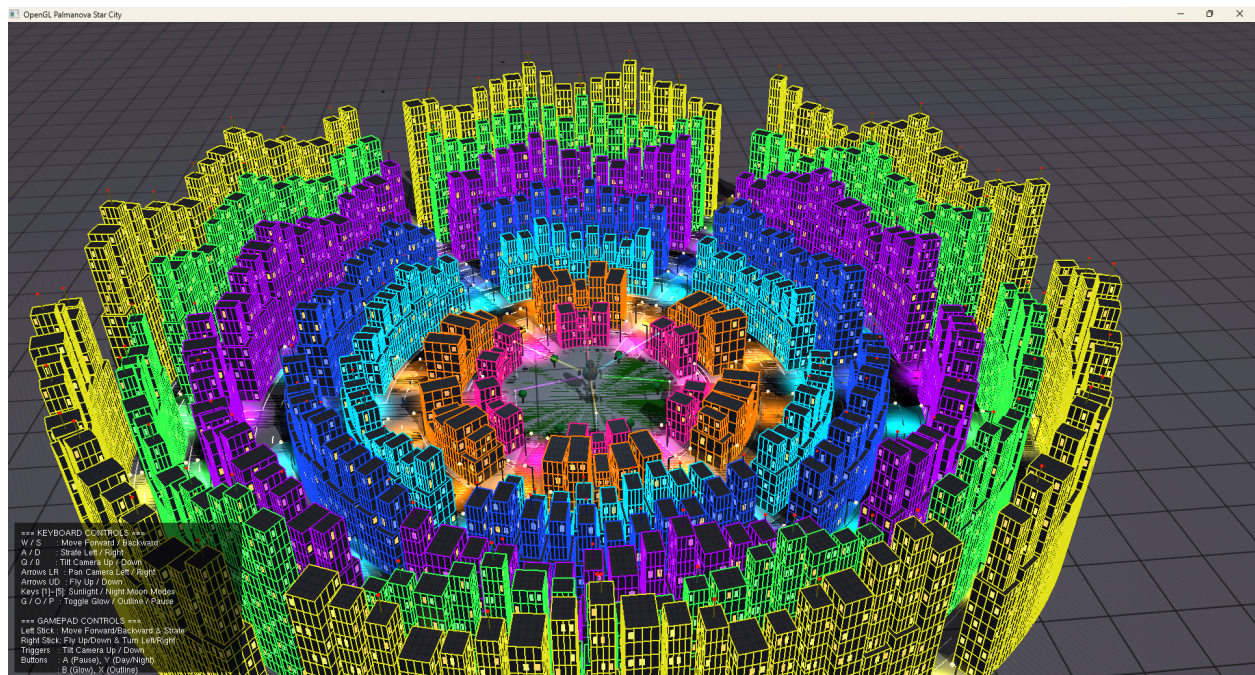
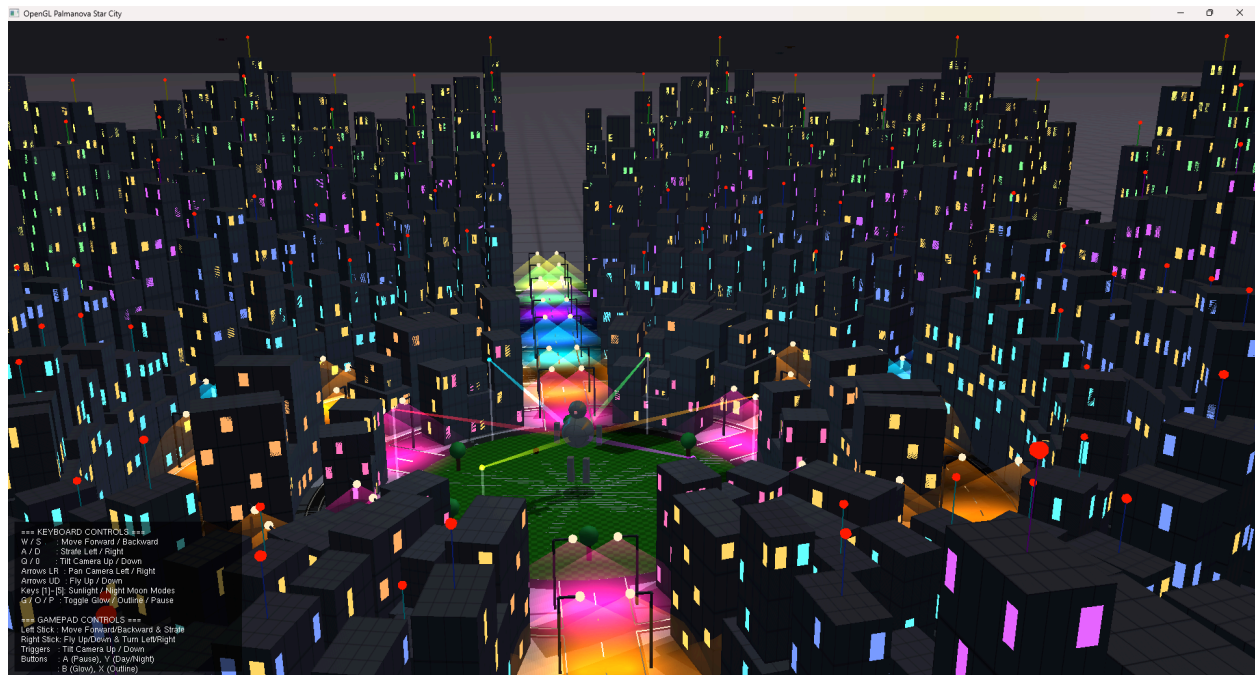
Keyboard Navigation

Key	Action
W / S	Move Forward / Backward.
A / D	Strafe Left / Right.
O / Q	Tilt Camera Up / Tilt Camera Down.
Left Arrow / Right Arrow	Pan Camera Left / Right.
Up Arrow / Down Arrow	Fly Vertically Up / Down.
1 - 4	Activate Night Mode & switch between 4 different Moon positions.
5	Activate Daylight Mode (Bright Sunlight).
G	Toggle Skyscraper Windows & Spire Glow (Night only).
O	Toggle Skyscraper Neon Grid Outlines (Night only).
P	Toggle Pause / Play animations.

Gamepad Navigation (Xbox Controller) (Requires compiling with controller support enabled via make control or make ENABLE_CONTROLLER=1)

Control	Action
Left Thumbstick Y / X	Move Forward / Backward and Strafe Left / Right.
Right Thumbstick Y / X	Fly Vertically Up / Down and Turn Left / Right.
Left & Right Triggers	Tilt Camera Up / Down.
Button A	Toggle Animation Pause.
Button Y	Toggle Day / Night Mode.
Button X	Toggle Skyscraper Glow.
Button B	Toggle Skyscraper Outlines.

iii. Screenshots



iv. Requirement Checklist

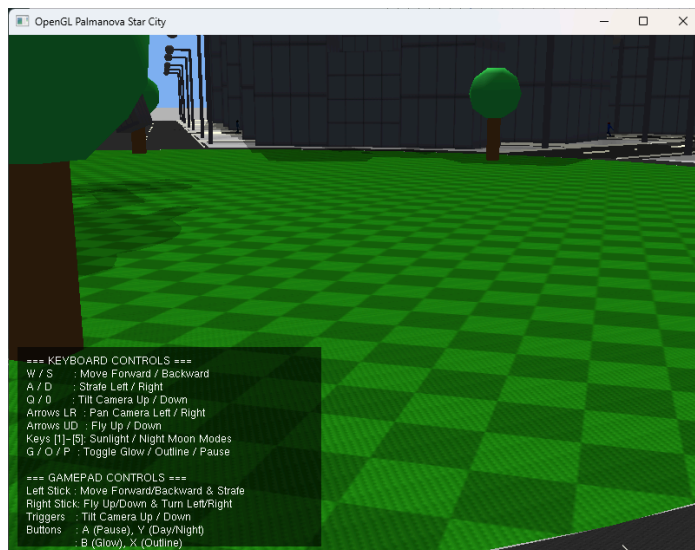
Below is the clear identification of which objects fulfill the project requirements. *(Note: Please insert screenshot evidence beneath each item before submitting).*

- **1. Texture Mapping (10%):**

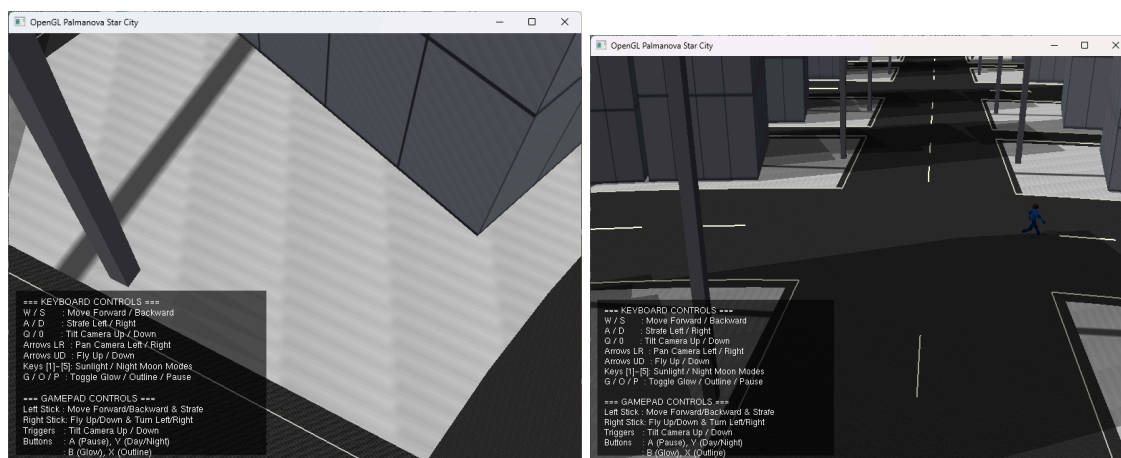
The project utilizes 7 custom-generated textures, applying them to various object types.

The textures include cyber grass, concrete, asphalt, metal, neon grids, holograms, and spotlights.

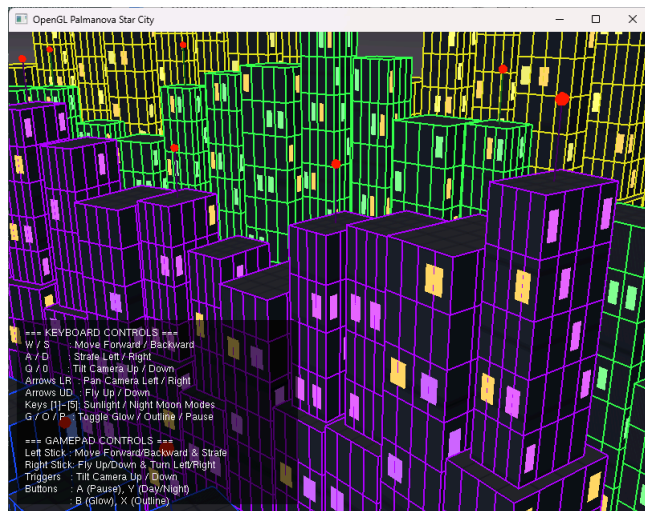
The AI-related objects (pedestrians and drones) fulfill the specific AI texture requirement by using the hologram.tga and cyber_metal.tga textures.



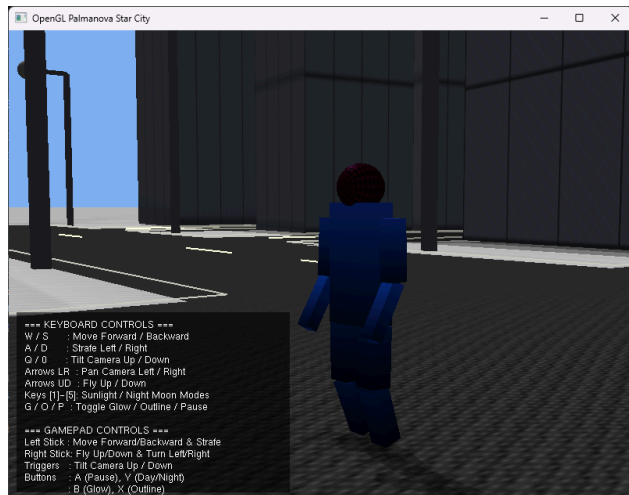
Texture: grass



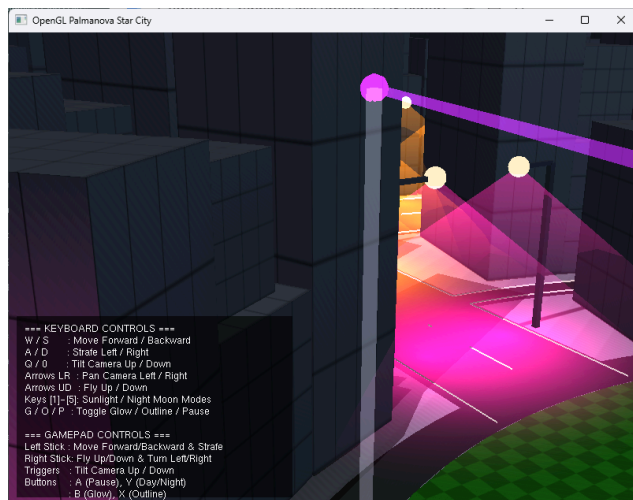
Texture: concrete, metal and asphalt



Texture: neon grid



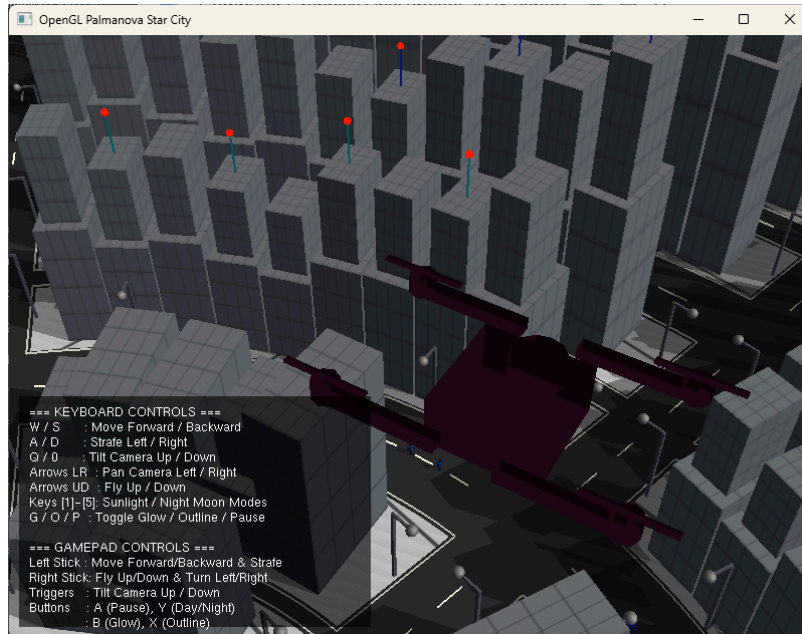
Texture: hologram



Texture: Spotlight

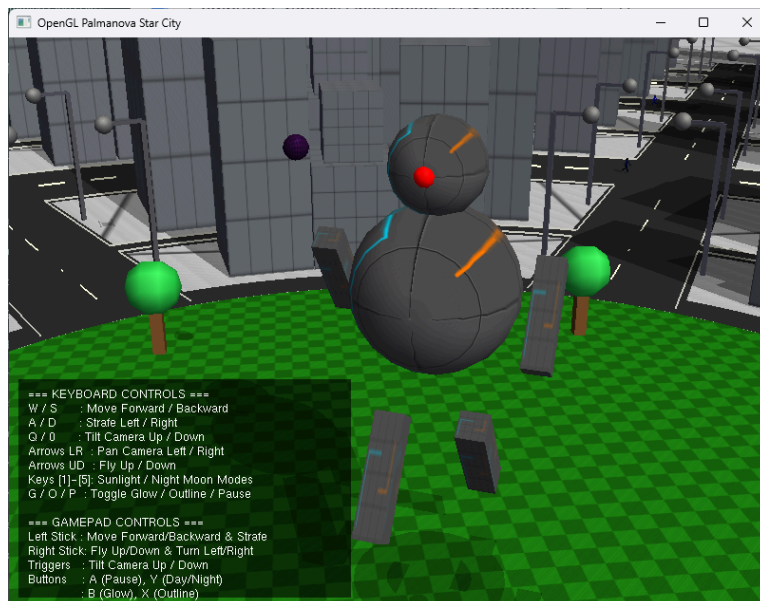
2. AI theme-related Object Model (10%):

- The project loads a custom external drone.obj model, representing an AI delivery drone.
- There are 8 active drones rendered in the scene, which include altitude oscillation and body spin animations.



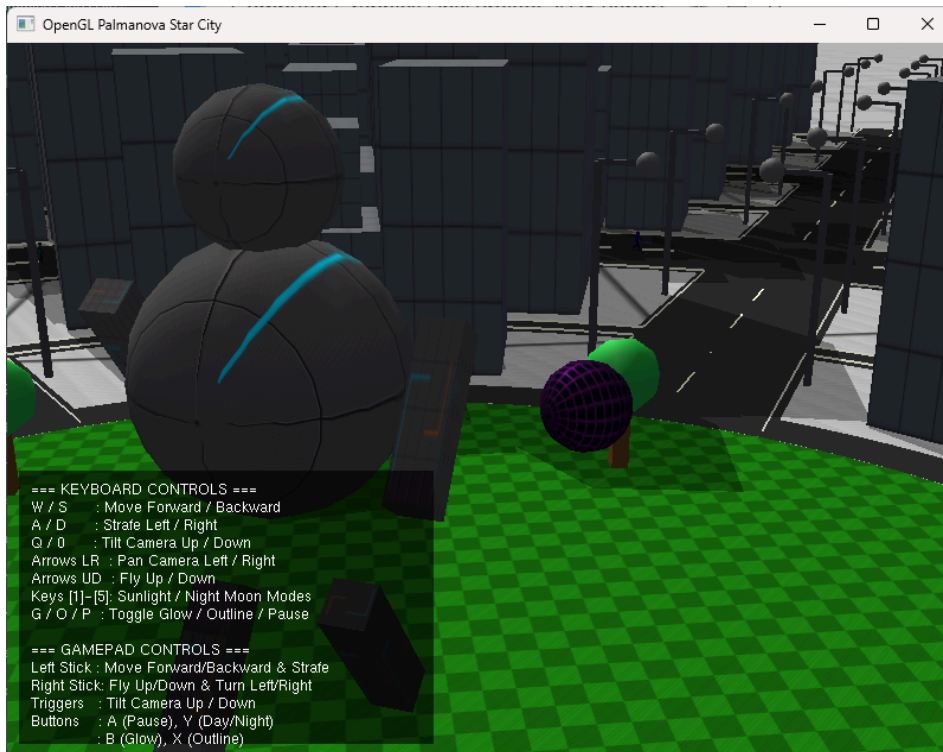
• 3a. Self Rotation (5%):

- Object_A is a central giant humanoid robot located at the park center.
- It fulfills this requirement by rotating continuously on its Y-axis.



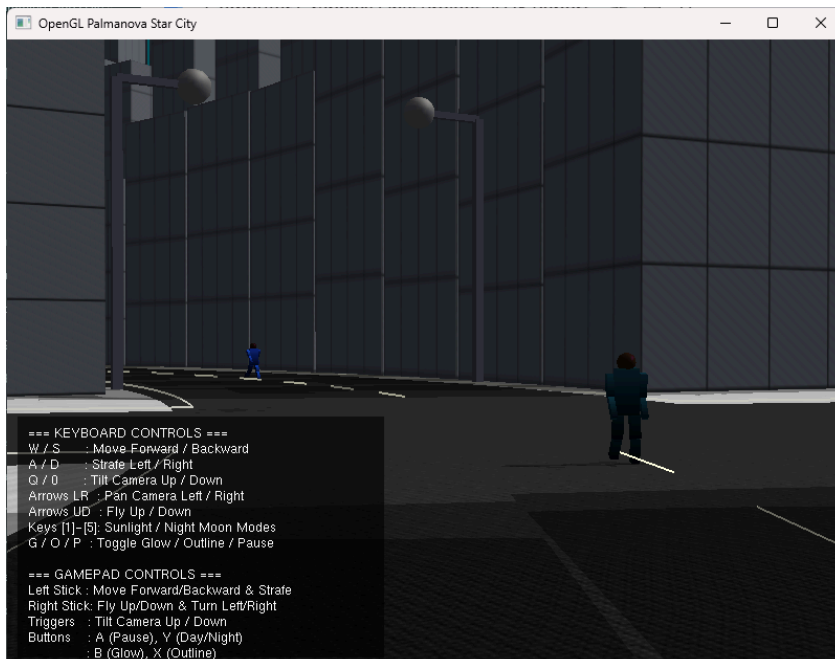
- **3b. Orbit Motion (5%):**

- Object_B is an orange surveillance satellite sphere.
- It revolves around the central robot (Object_A) while oscillating vertically using a sine wave function.

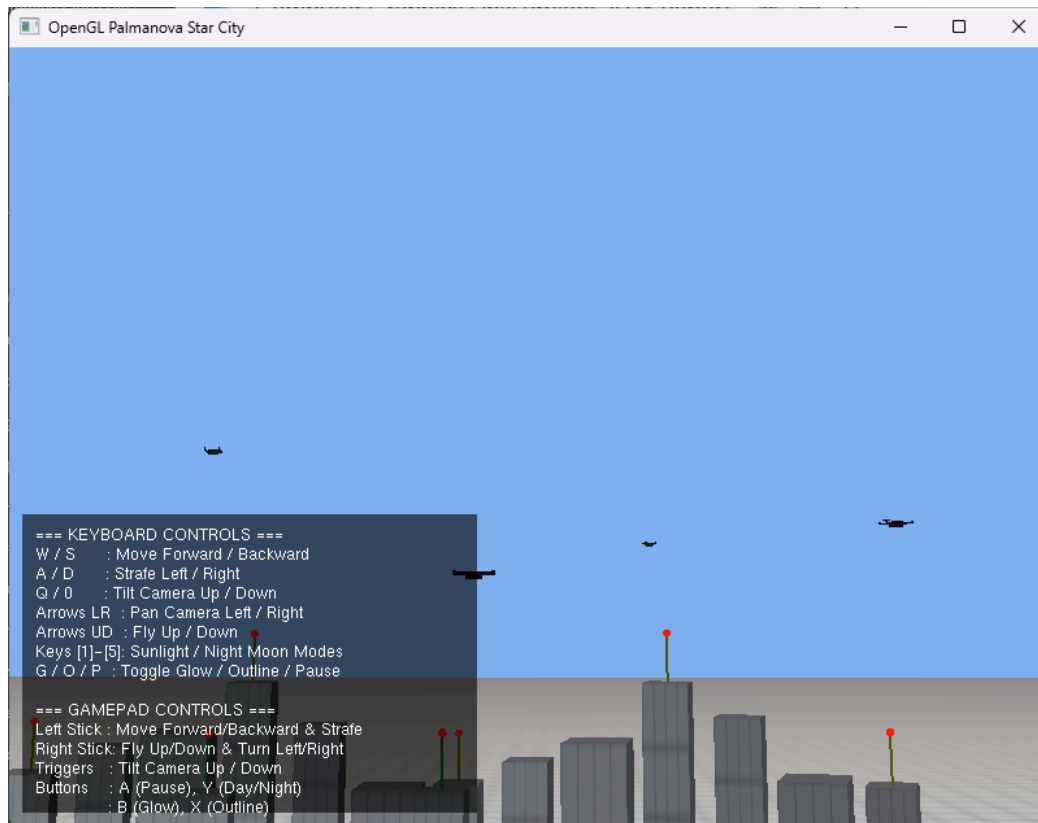


- **3c. Custom Animation (10%):**

- **Animation 1:** 30 robot pedestrians utilize hierarchical walking, swinging legs and arms in counter-phases using sine/cosine functions.

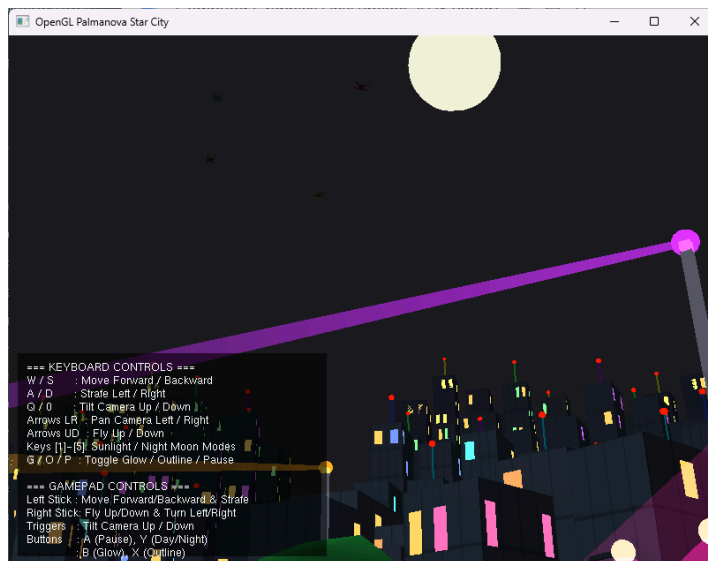


- **Animation 2:** 8 AI delivery drones follow dynamic 3D orbital paths using sine and cosine waves.

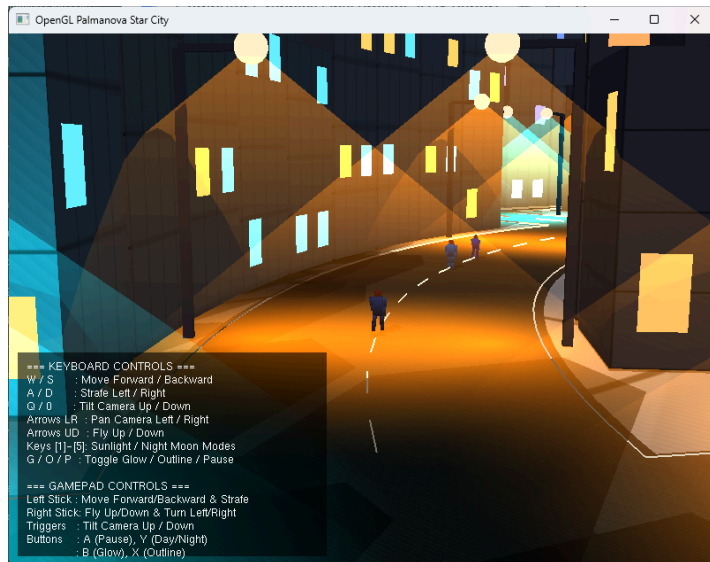


- **4. Light and Shadow (10%):**

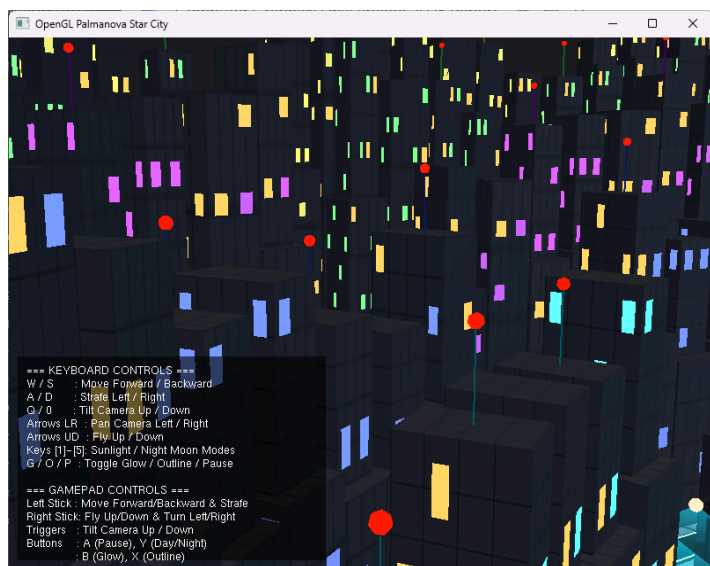
- The project implements 7 distinct hardware light sources, including global directional light, a neon point light, and dynamic street spotlights.
- Visible planar shadows are rendered using matrix transformations.



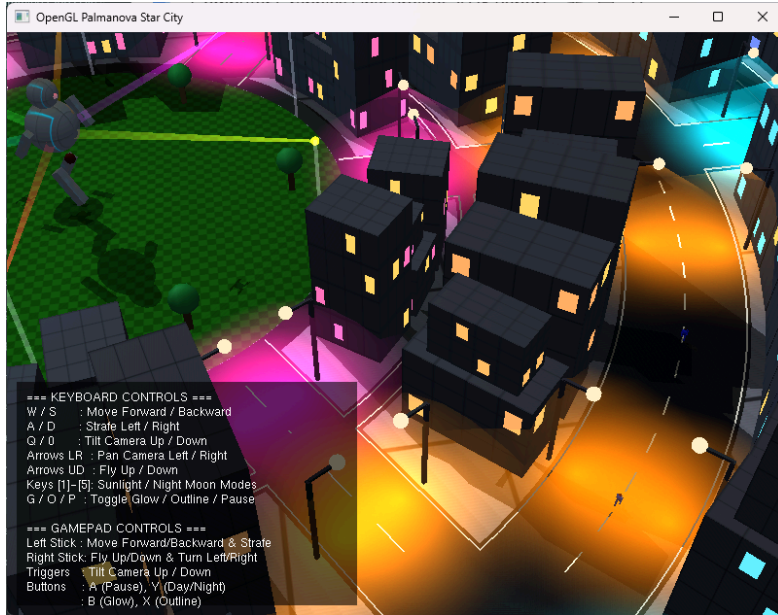
Moon Light: Directional light



Street Lamp: Spotlight



Neon: Point light



Shadows

v. Technical Challenges

This section outlines the difficulties encountered during development and the solutions adopted.

- **Challenge 1: Z-Fighting (Flickering Pavements)**
 - *Difficulty:* The ground terrain, roads, and avenues were all placed at the exact same height ($Y = -0.4$), causing the GPU depth buffer to struggle and create severe rendering flickering.
 - *Solution:* Implemented microscopic height offsets to layer the objects cleanly. Base concrete was set to $Y = -0.4$, road strips to $Y = -0.398$, road markings to $Y = -0.395$, and skyscraper bases to $Y = -0.39$ to eliminate visual glitches.
- **Challenge 2: Multi-Overlay Shadow Darkness (Double Blending)**
 - *Difficulty:* Overlapping 3D objects projecting planar shadows onto the same ground plane resulted in pitch-black, multiply-blended overlap regions. Additionally, an incorrect stencil write-mask configuration (`glStencilOp(GL_INCR, GL_INCR, GL_INCR)`) caused stencil values to increment on stencil-test failure, making overlapping layers alternate draw states and flicker frame-by-frame.
 - *Solution:* Integrated the Stencil Buffer. We corrected the stencil update rule to `glStencilOp(GL_KEEP, GL_KEEP, GL_INCR)`. Now, if the stencil test fails, the stencil value is kept (`GL_KEEP`) instead of incrementing, ensuring that overlapping shadow fragments are safely discarded and rendering is stable and flicker-free.
- **Challenge 3: Hardware Light Source Limit**
 - *Difficulty:* Standard OpenGL limits applications to only 8 hardware lights, which was not enough for a massive city requiring 180 streetlights. Additionally, when sorting the static database of 180 lamps in-place, equal-distance concentric lamps would

- constantly shuffle order every frame, causing the active streetlights to flicker rapidly.
- *Solution:* Developed a Dynamic Spotlight Allocation System. At every frame, the program calculates the distance from the player camera to all 180 lamps. To prevent flickering, we copy the database to a local array before sorting, leaving the static array structure stable. The program then dynamically binds the 4 closest lamps to hardware channels without order fluctuations.
 - **Challenge 4: Backface Culling & Winding Order**
 - *Difficulty:* The circular central park grass disc became invisible from certain viewing angles because the vertices were drawn in a clockwise order.
 - *Solution:* Rewrote the rendering loop with a negative angle step to produce a counter-clockwise (CCW) winding order, properly satisfying the backface culling requirements.
 - **Challenge 5: Depth Buffer Z-Fighting & Flickering at High Altitudes**
 - *Difficulty:* When flying the camera very high, the ground pavement and road asphalt (separated by only 0.002 units) suffered from severe Z-fighting (material tearing) because a very close near clipping plane (0.01f) compressed the 24-bit depth precision at large distances. Additionally, the ground and Moon clipped and popped as they exceeded the far clipping plane (300.0f).
 - *Solution:* We adjusted the near clipping plane to 0.3f in gluPerspective, redistributing the depth buffer resolution to be 30x more precise at far distances. We increased the far clipping plane to 600.0f and clamped the camera's flying altitude between 0.0f and 120.0f, ensuring all terrain remains cleanly layered and fully within the rendering boundaries.